



HOCHSCHULE
HAMM-LIPPSTADT

Mixed Criticality Scheduling

Name	Nader Yahia
Matriculation	2210026
Supervisor	Prof. Dr.rer.nat. Stefan Henkler

Motivation

1. Coexistence of Different Criticality Levels

Modern embedded systems increasingly integrate functions with different levels of safety and importance on the same hardware platform:

Examples:

- Automotive: Braking + Infotainment
- Avionics: Flight Control + Passenger Services
- Medical: Life-Support + Monitoring

The Challenge:

Guarantee deadlines for high-criticality tasks while efficiently utilizing system resources.

Motivation

2. Classical RTS Pitfalls

Fixed Priority (FP)

- Static priorities assigned offline
- Simple and predictable
- Limited processor utilization [1]

Earliest Deadline First (EDF)

- Dynamic priorities based on deadlines
- Optimal on a uniprocessor
- Can achieve full processor utilization [2]

- Standard EDF treats all tasks equally.
- EDF cannot distinguish task criticality.
- HI-criticality deadlines may be jeopardized.

Can EDF be adapted to favor HI-criticality tasks before an overrun occurs?



Motivation

3. Single WCET Failure in Practice

Task	Criticality	WCET (measured)	WCET (safe bound)	T
τ_1 (Braking)	High	3 ms	5 ms	20 ms
τ_2 (Control)	High	4 ms	6 ms	20 ms
τ_3 (Infotainment)	Low	2 ms	10 ms	20 ms

- Total Utilization (Worst-Case Design):

$$U = \frac{5}{20} + \frac{6}{20} + \frac{10}{20} = 1.05 \quad \longrightarrow \quad \text{System becomes unschedulable}$$

- Actual Execution Behavior:

$$U_{avg} = \frac{3}{20} + \frac{4}{20} + \frac{2}{20} = 0.45 \quad \longrightarrow \quad \text{System is actually heavily under-utilized}$$

- Worst-case design ensures safety infrequent **overloads**, but is often resource-inefficient in **normal operation**.

The Vestal Model

- Using a single WCET for all tasks forces the system to be dimensioned using pessimistic execution-time bounds.

$$[U_{naive} = \sum_{\tau_i \in T} \frac{C_i(HI)}{T_i}] \quad [3]$$

Vestal's Idea:

- Two execution assumptions are maintained **LO-mode** and **HI-mode**

LO-mode

All tasks meet deadlines.

HI-mode

- All HI meet deadlines
- LO tasks suspended

- HI tasks have two WCET bounds:
 - $C_i(LO)$ for normal operation.
 - $C_i(HI)$ for worst-case execution.

Mode Switching Concept

- Normal Operation (LO-Mode):
 - All tasks execute using ($C_i(LO)$).
 - LO- and HI-criticality tasks are scheduled.
 - System assumes nominal execution behavior

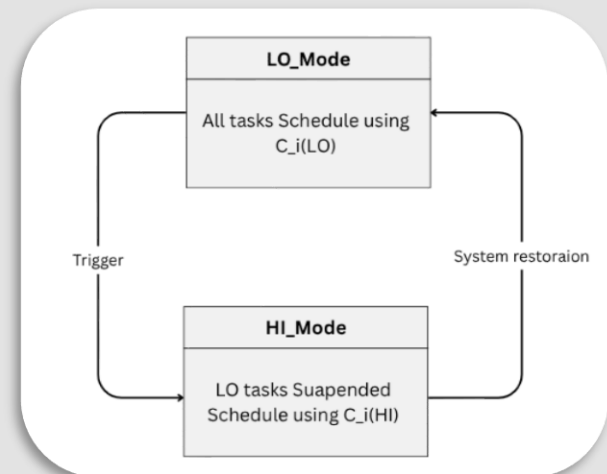
- Mode Switch Trigger:

- A HI-criticality task exceeds its LO-mode budget

$$[e_k(\tau_i) > C_i(LO)] \quad [3]$$

- HI-Mode Operation:

- System transitions to HI-mode.
 - HI-criticality tasks receive execution up to ($C_i(HI)$).
 - LO-criticality tasks may be suspended.
 - Scheduler prioritizes maintaining HI-criticality guarantees.



EDF with Virtual Deadlines (EDF-VD)

Main Idea

- Give HI-criticality tasks earlier scheduling deadlines:

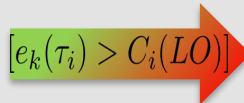
$$\hat{D}_i = \begin{cases} xD_i, & L_i = HI \\ D_i, & L_i = LO \end{cases} \quad x \in (0, 1] \quad [5]$$

LO-Mode

All HI and LO tasks execute.

HI tasks use compressed (VD).

Execution times $C_i(LO)$ used.



HI-Mode

when a HI task exceeds $C_i(LO)$

LO tasks (suspended), HI continue.

true deadlines D_i and uses $C_i(HI)$

- EDF-VD preserves HI-criticality guarantees while improving utilization compared to naive EDF.

Tradeoffs

1. Safety vs Efficiency

- Increasing HI protection reduces available LO execution time.
- Decreasing protection risks HI-task deadline misses.

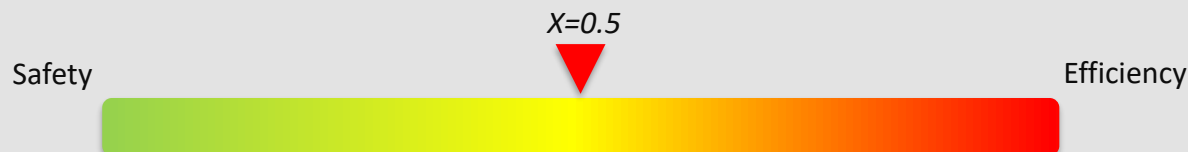
Safety-First	Efficiency-First
HI task: C(LO)=4 ms , C(HI)=10 ms.	HI task: C(LO)=4 , C(HI)=10 ms.
Scheduler reserves full 10 ms. (6ms)	Scheduler allocates only 4 ms.
HI-task deadlines guaranteed.	More processor time available for LO.
High safety, lower resource utilization.	High utilization, lower safety margin.

Role of the Compression Factor x

- **Small x :** higher HI safety, lower LO capacity.
- **Large x :** higher LO efficiency, lower HI safety margin.

$$B_{LO} = 1 - \frac{U_{HI}^{LO}}{x}$$

[5]



Tradeoffs

2. WCET Pessimism vs Utilization

- HI WCETs are conservative by design.
- Actual execution is typically much smaller.
- Leads to unused reserved capacity (“pessimism tax”).

Criticality Augmentation Factor

$$\rho = \frac{C(HI)}{C(LO)} \quad [5]$$

- Measures how larger the certified WCET is than the nominal WCET.
- Industrial benchmarks range from 1.5× to 10×.

pi (Aug.Factor)	Utilization	LO bandwidth loss
1.5×	1.5×	33% of HI-reserved bandwidth wasted
3×	3×	67% of HI-reserved bandwidth wasted
5×	5×	80% of HI-reserved bandwidth wasted
10×	10×	90% of HI-reserved bandwidth wasted

Tradeoffs

2. WCET Pessimism vs Utilization

EDF-VD balances this via compression factor x .

- We have one HI-criticality task:

- Period = 10 ms
- $C(LO)=2$ ms
- $C(HI)=6$ ms



$$\rho = \frac{C(HI)}{C(LO)} = \frac{6}{2} = 3$$

- HI-task utilization under LO assumptions

$$U_{HI}^{LO} = \frac{2}{10} = 0.2$$

- HI utilization reserved

$$U_{HI}^{HI} = \frac{6}{10} = 0.6$$

- Unused Reserved Fraction

$$1 - \frac{U_{HI}^{LO}}{U_{HI}^{HI}} = 1 - \frac{0.2}{0.6} = 0.667 \approx 66.7\%$$

Tradeoffs

3. LO-Task Degradation

- When a HI-criticality task exceeds its LO budget:
 - System enters HI-mode
 - LO-criticality tasks are suspended
 - Processor capacity is reserved for HI-criticality tasks
 - HI-task deadlines remain protected

Example

$$H = 1000 \text{ ms}, t_s = 200 \text{ ms}$$

$$\text{LO Starvation} = H - (t_s \bmod H) \quad \longrightarrow \quad 1000 - 200 = 800 \text{ ms}$$

Therefore, LO tasks receive **no service for 800 ms**.

LO Availability

$$\alpha_{LO} = \frac{\text{Time spent in LO-mode}}{H} = \frac{200 \text{ ms}}{1000 \text{ ms}} = 0.2$$

$$\left. \begin{array}{l} \alpha_{LO} \approx 1 \\ \alpha_{LO} \rightarrow 0 \end{array} \right\} \begin{array}{l} \text{Normal} \\ \text{Operation} \\ \\ \text{Frequent} \\ \text{Overruns} \end{array} \quad [7]$$

Limitations

- Requires correct WCET bounds (non-trivial in practice)[6].
 - Hard to guarantee in modern architectures
 - Cache hit or miss & speculation affect validity
- LO tasks may be suspended for long durations [3].
- No formal LO-service guarantees in HI-mode [3].
- Theory assumes a single-core processor [4].
 - Multicore introduces shared resource interference.
 - WCET and schedulability guarantees may break.
- Practical scalability limits [4]
 - Multiple criticality levels increase complexity.
 - Formal analysis becomes computationally expensive.

Conclusion

- Mixed-Criticality Scheduling provides a formal framework to handle systems with heterogeneous safety requirements under shared computational resources.
- Core system behavior is governed by a fundamental tension between: Safety guarantees vs. resource efficiency WCET pessimism vs. practical utilization HI-task correctness vs. LO-task service degradation
- The analysis shows that system correctness is achieved through mode separation and controlled LO-task suspension, rather than uniform guarantees across all tasks.
- However, key limitations remain in: WCET uncertainty and hardware dependence LO-task starvation under sustained overload Extension challenges for multicore systems

References

- [1] C. L. Liu and J. W. Layland, “Scheduling algorithms for multiprogramming in a hard-real-time environment,” *J. ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973, doi: 10.1145/321738.321743.
- [2] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 4th ed. Cham, Switzerland: Springer, 2020.
- [3] S. Vestal, “Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance,” in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, 2007, pp. 239–243.
- [4] S. Baruah, A. Burns, and R. I. Davis, “Response-time analysis for mixed-criticality systems,” in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, 2011.
- [5] S. Baruah, V. Bonifaci, G. Buttazzo, and A. Marchetti-Spaccamela, “Mixed-criticality scheduling of sporadic task systems,” in *Proc. Euromicro Conf. Real-Time Systems (ECRTS)*, 2011, pp. 1–12.
- [6] R. Wilhelm *et al.*, “The worst-case execution-time problem—overview of methods and survey of tools,” *ACM Transactions on Embedded Computing Systems*, vol. 7, no. 3, pp. 1–53, 2008, doi: 10.1145/1347375.1347389.

References

[7] 2210026, “AI Usage Protocol for All Milestones: Mixed Criticality Scheduling,” 2026. [Online]. Available: [ai_usage_protocol_2210026_All_Milestones.json](#). Accessed: 2026.

Q & A

Assumptions

Assumption	Name	Description
A1	Single-Core Processor	All tasks execute on a single processor.
A2	Preemptive Scheduling	Tasks can be interrupted at any time.
A3	Deterministic WCET Bounds	$(C_i(LO))$ and $(C_i(HI))$ are assumed to be safe upper bounds.
A4	Periodic/Sporadic Tasks	Tasks are released periodically or with a minimum inter-arrival time (T_i).
A5	Two-Level Criticality	Tasks are classified only as LO or HI criticality.
A6	Negligible Overhead	Scheduling and mode-switch overhead are assumed small compared to execution times.

Assumptions

Assumption	Name	Description
A1	Single-Core Processor	All tasks execute on a single processor.
A2	Preemptive Scheduling	Jobs may be preempted at any instant.
A3	Task Independence	Tasks are independent and do not share synchronization objects.
A4	Deterministic WCET Bounds	The values $(C_i(LO))$ and $(C_i(HI))$ are assumed to be safe upper bounds.
A5	Periodic/Sporadic Activation	Tasks are released periodically or sporadically with minimum inter-arrival time (T_i) .
A6	No Resource Sharing	No semaphores, mutexes, or shared I/O resources are considered.
A7	Instant Mode Switch Detection	Mode switch from LO to HI is assumed to be detected without delay.
A8	Negligible Runtime Overhead	Scheduling, monitoring, and context-switch overheads are assumed small compared to WCET.
A9	Two-Level Criticality Model	Tasks are classified only into LO and HI criticality levels.

System Model

- The system consists of a set of sporadic/periodic tasks:

$$T = \{\tau_1, \tau_2, \dots, \tau_n\}$$

- Each τ_i is defined as:

$$\tau_i = (T_i, D_i, C_i(LO), C_i(HI), L_i)$$

T_i		<i>period (minimum inter-arrival time between task releases).</i>
D_i		<i>relative deadline, with $D_i \leq T_i$.</i>
$C_i(LO)$		<i>execution time under low-assurance analysis.</i>
$C_i(HI)$		<i>execution time under high-assurance analysis.</i>
$L_i \in \{LO, HI\}$		<i>task criticality level. [4]</i>

- Each high-criticality task is modeled with two execution budgets:
 - optimistic (LO) estimate.
 - conservative (HI) certification-bound estimate.
- Analyze how mixed-criticality scheduling maintains safety under execution overruns while improving processor utilization.

Note

- Same phenomenon appears at different abstraction levels in real-time systems: scheduling policy, execution-time analysis, numerical utilization, and runtime behavior. Each slide isolates one of these perspectives to avoid mixing concepts in a single view.
 - The first slide focuses on the **design-level** tradeoff between safety and efficiency using the compression factor x .
 - The second slide explains the **analytical cause, namely WCET pessimism** and why reserved capacity is not fully used.
 - The third slide translates this into a quantitative utilization loss, showing how much capacity is wasted in practice.
 - The fourth we moved to the **runtime perspective**, where mode switching leads to LO degradation and guarantees remain only for HI-criticality tasks.”